

Sequence Diagrams

Henry Tang

hktang@ualberta.ca

Department of Computing Science
University of Alberta

CMPUT 301 – Introduction to Software Engineering

Slides adapted from Dr. Hazel Campbell, Dr. Ken Wong, and Dr. Abram Hindle

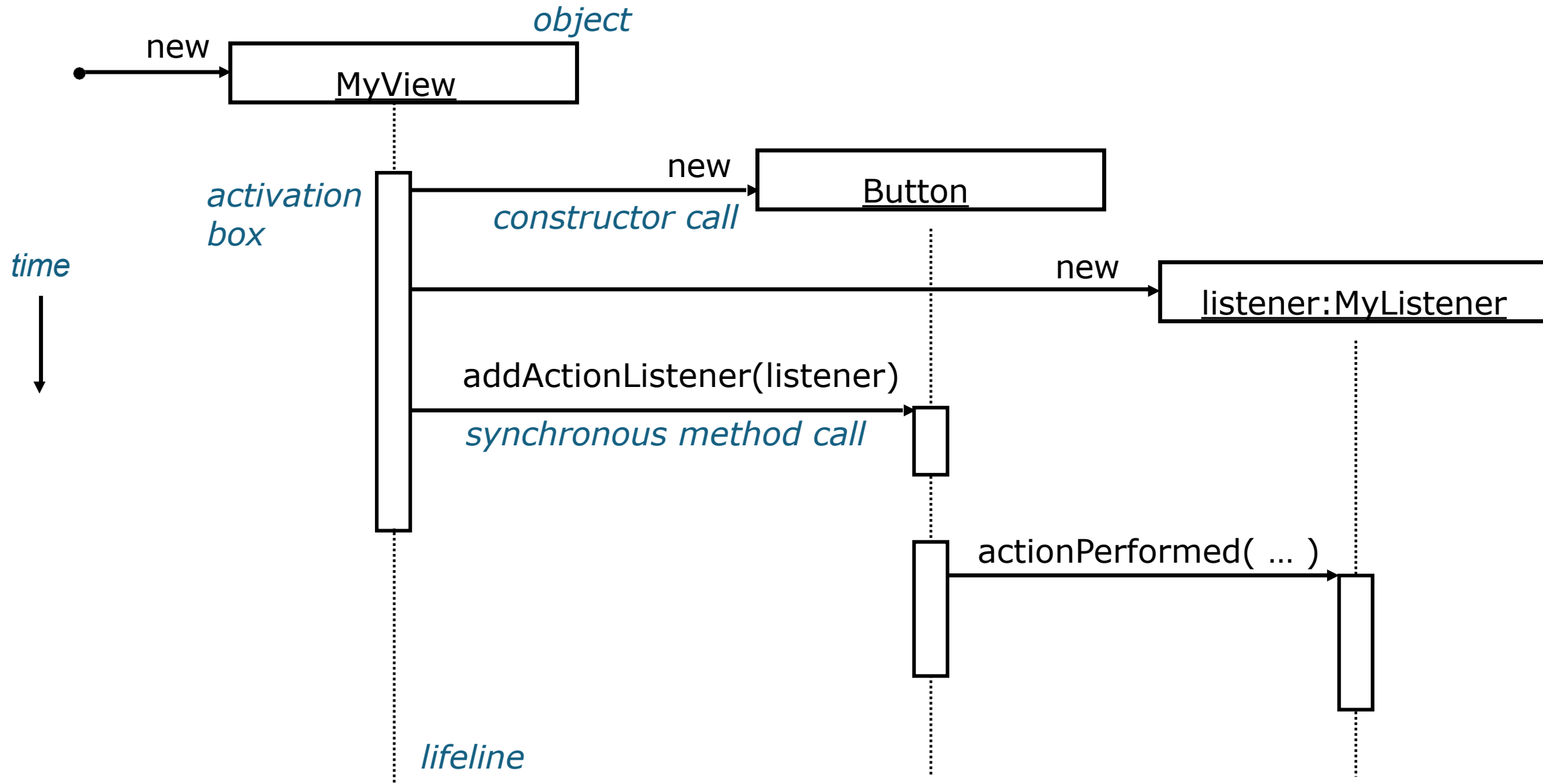


Updated by Michelle Deng 2026

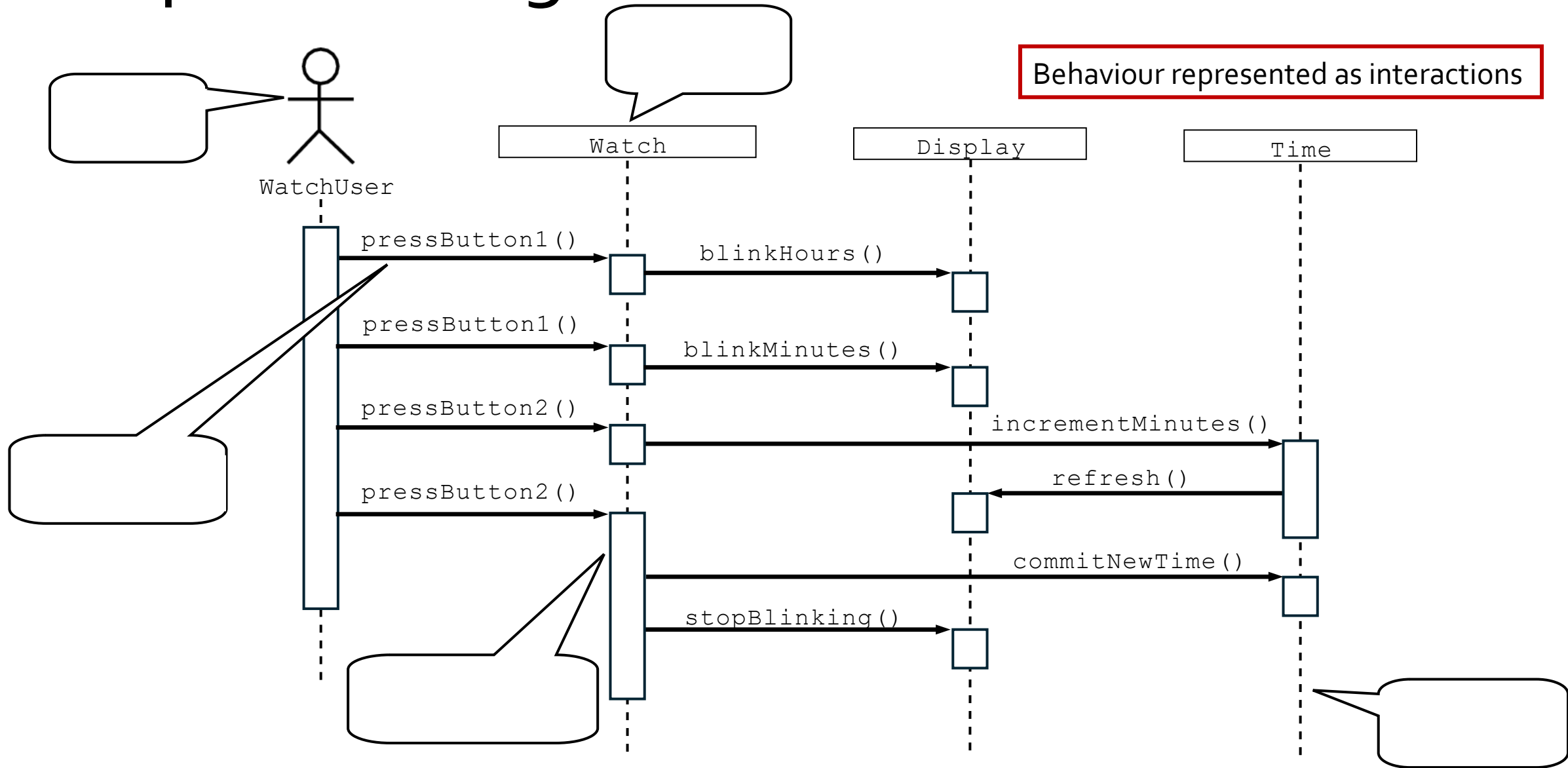
Table of Contents

- First Look at UML Sequence Diagram
- Basic Notation
- Use Cases
- More Notation

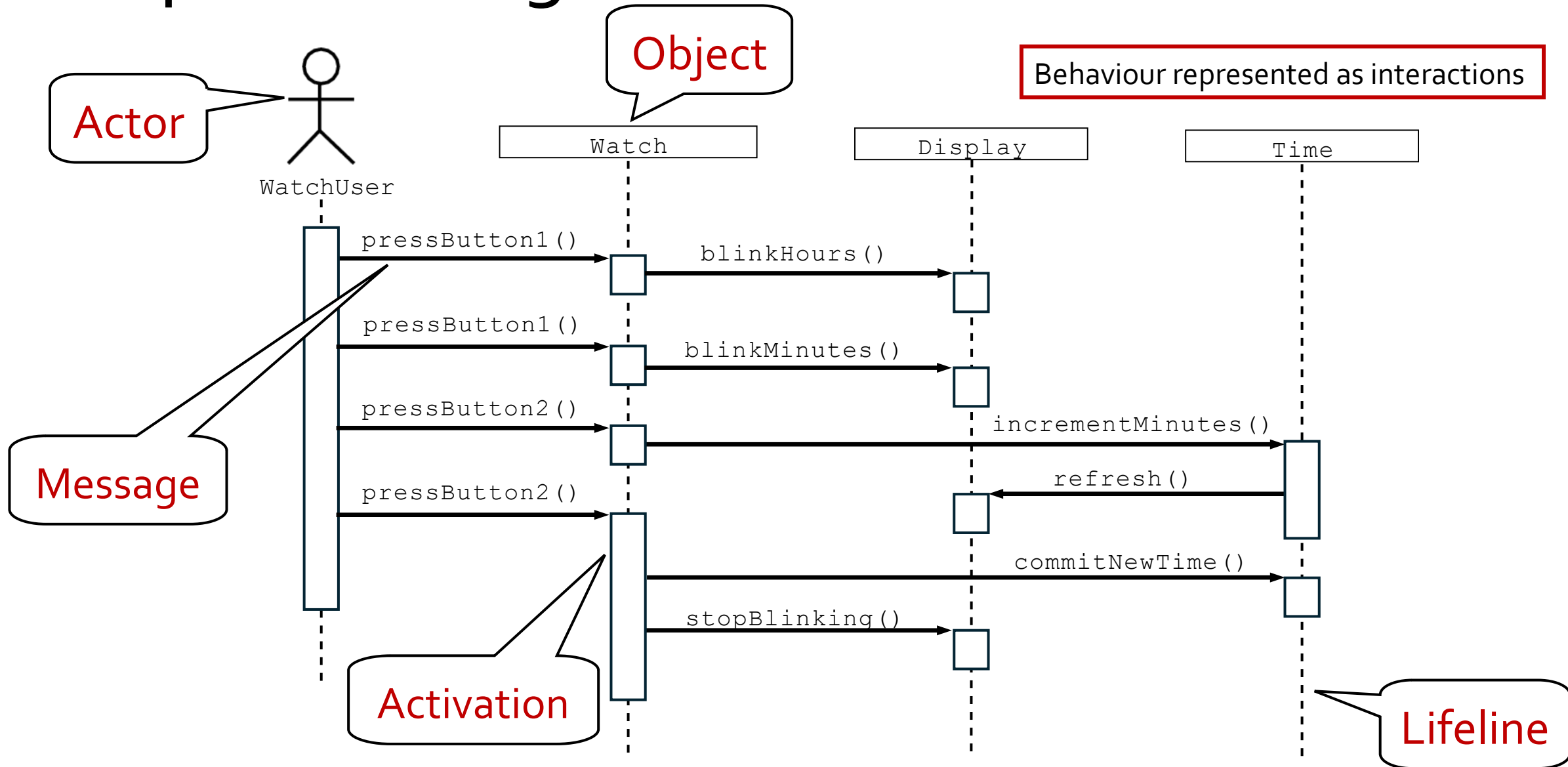
First Look: UML Sequence Diagram



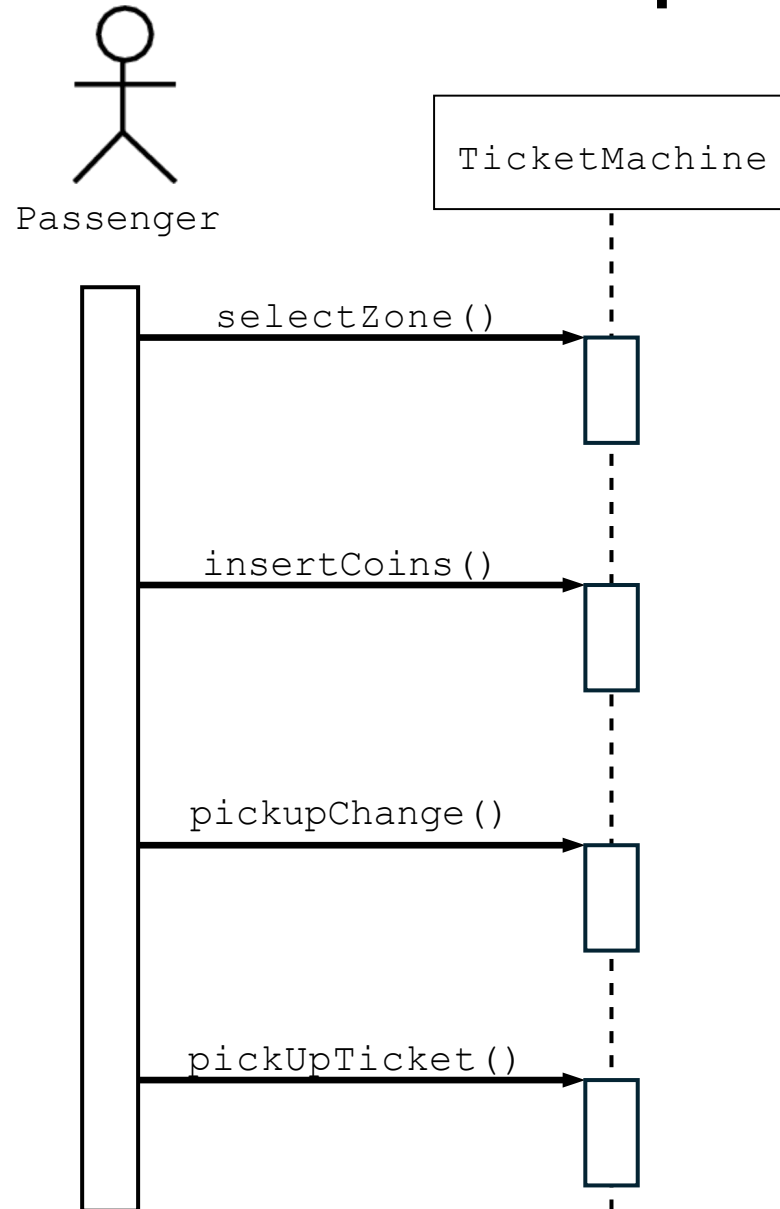
Sequence Diagram Basic Notation



Sequence Diagram Basic Notation

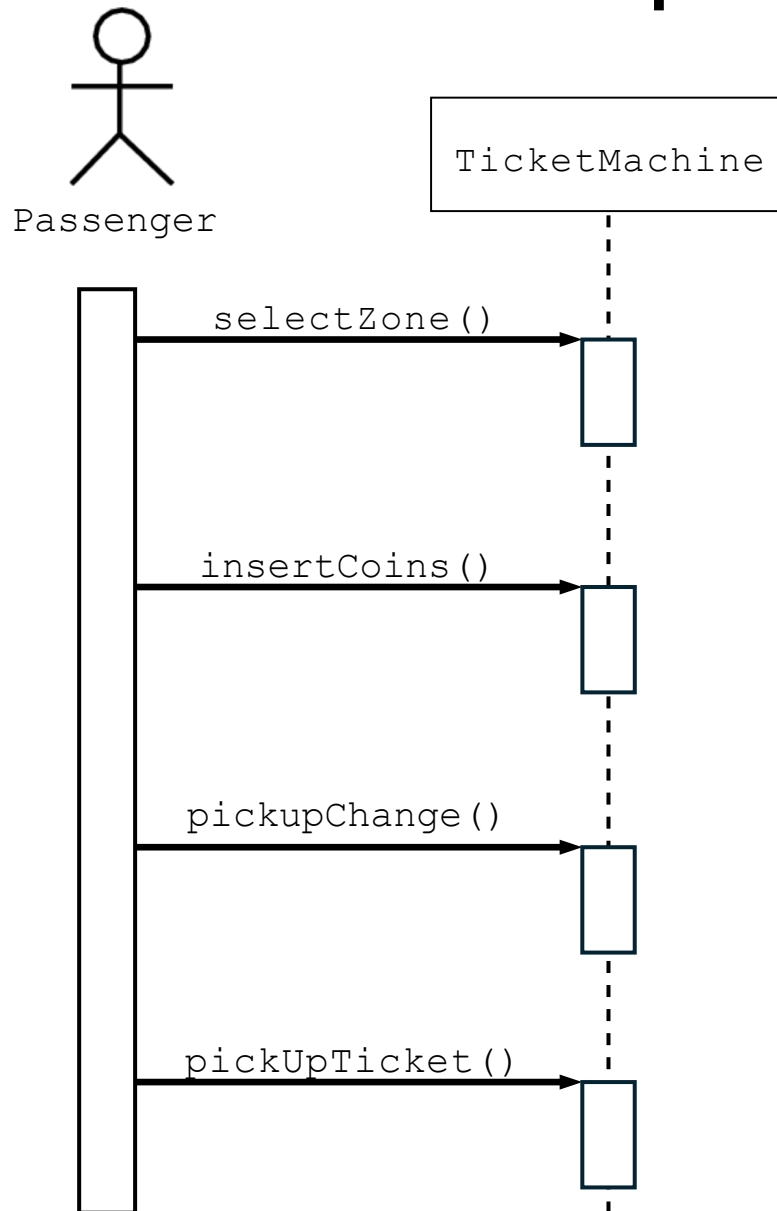


UML Sequence Diagrams Basic Notation



- Classes are represented by columns
- Messages are represented by arrows
- Activations are represented by narrow rectangles
- Lifelines are represented by dashed lines

UML Sequence Diagrams – Use Cases

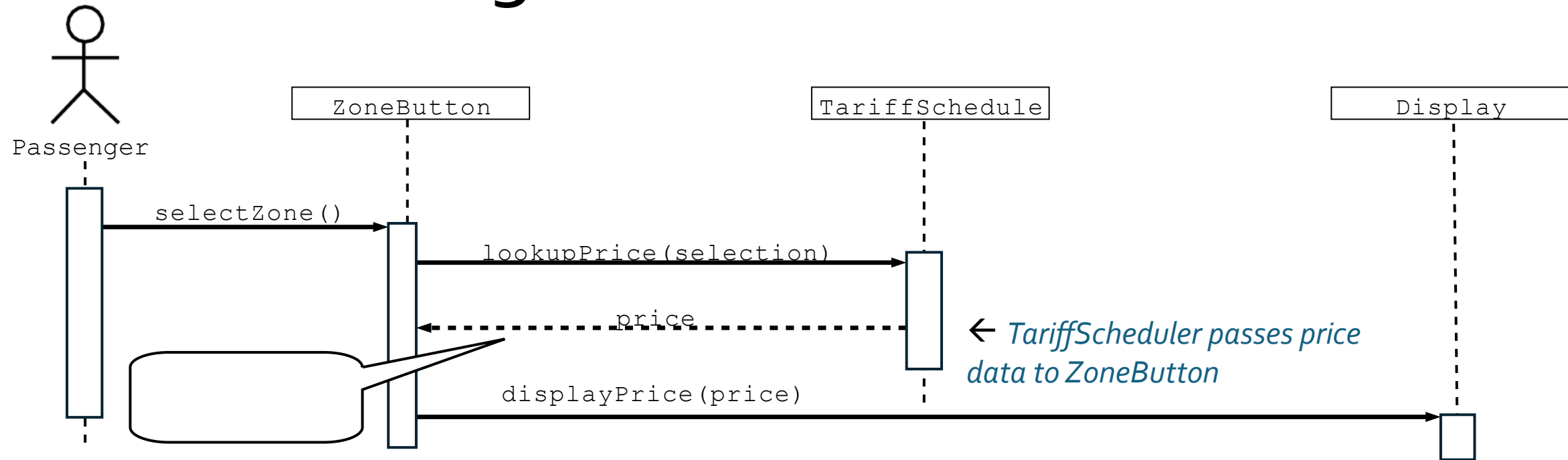


- Used during requirements analysis
 - To refine use case descriptions
 - To find additional objects (“participating objects”)
- Used during system design
 - To refine subsystem interfaces

More Notation

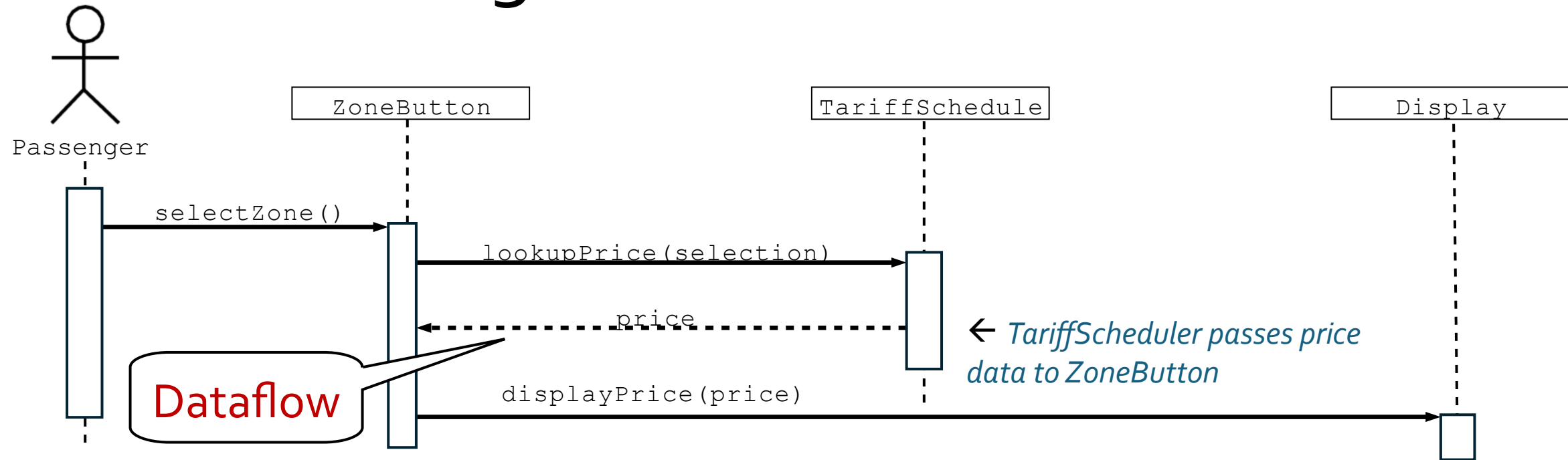
- We will now cover
 - Nested Messages and Data flow
 - Iteration and Condition
 - Creation and Destruction
 - Combined Fragments
 - Alternatives
 - Option
 - Loops

Nested Messages and Data Flow



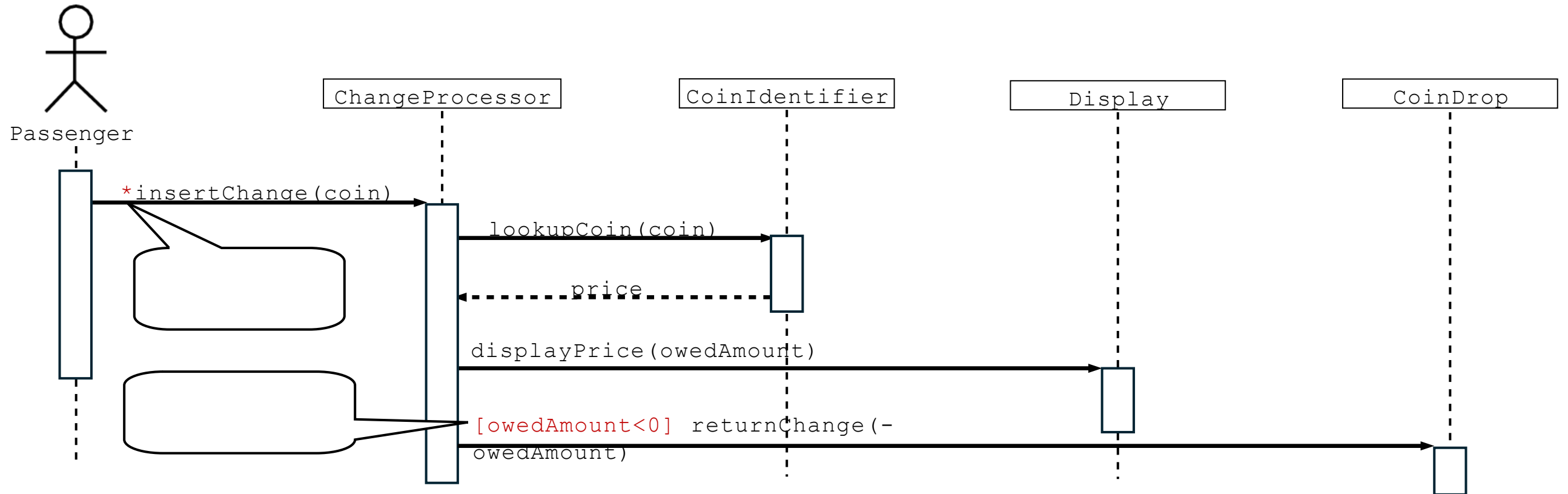
- The source of an arrow indicates the activation which sent the message
- An activation is as long as all nested activations
- Horizontal dashed arrows indicate data flow
- Vertical dashed lines indicate lifelines

Nested Messages and Data Flow



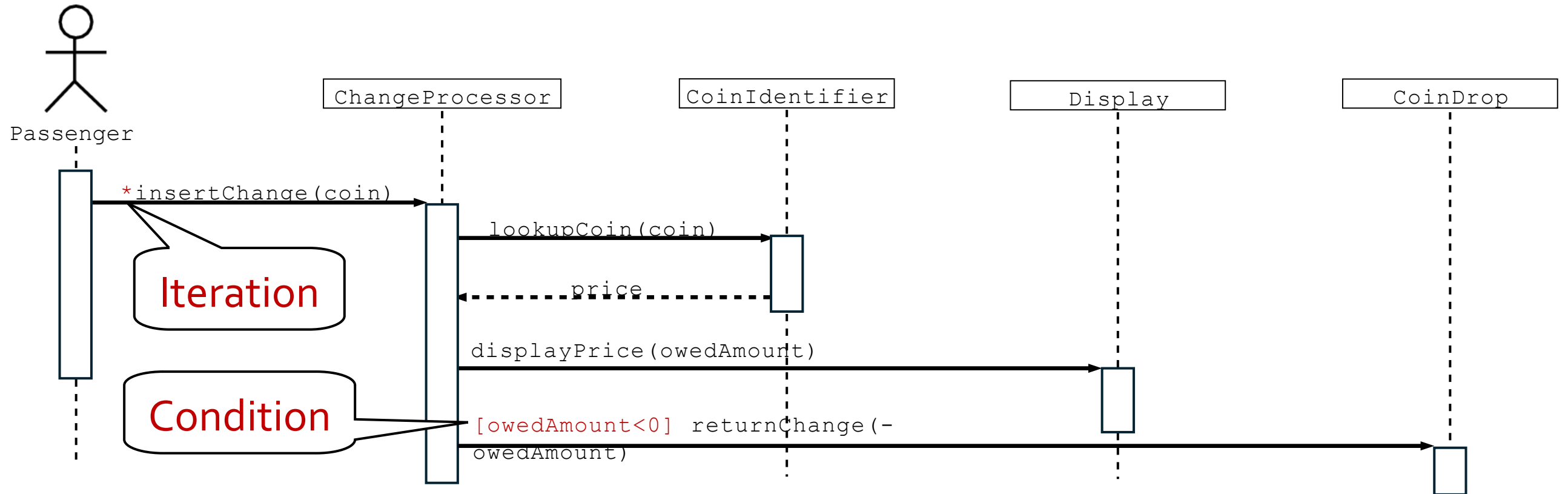
- The source of an arrow indicates the activation which sent the message
- An activation is as long as all nested activations
- Horizontal dashed arrows indicate data flow
- Vertical dashed lines indicate lifelines

Iteration and Condition



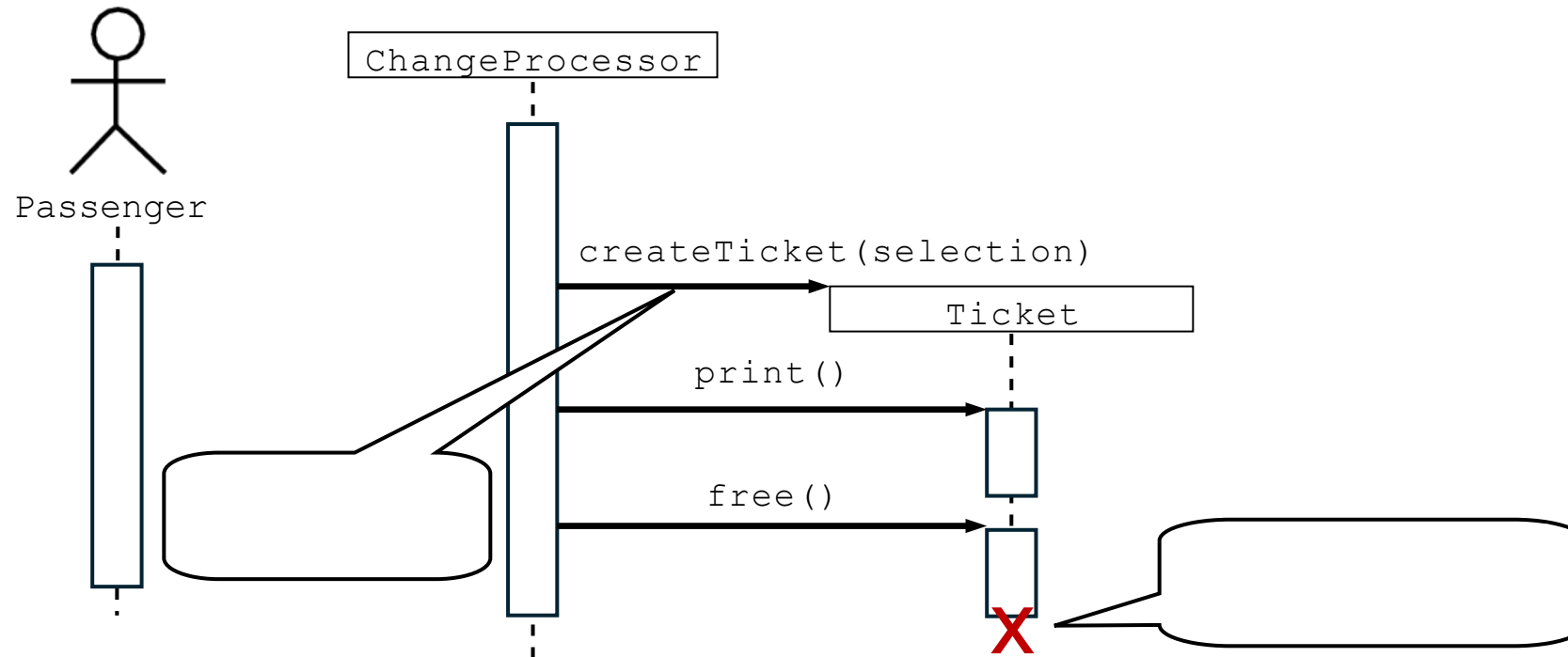
- Iteration is denoted by a * preceding the message name
- Condition is denoted by Boolean expression in [] before the message name
 - Also known as a guard

Iteration and Condition



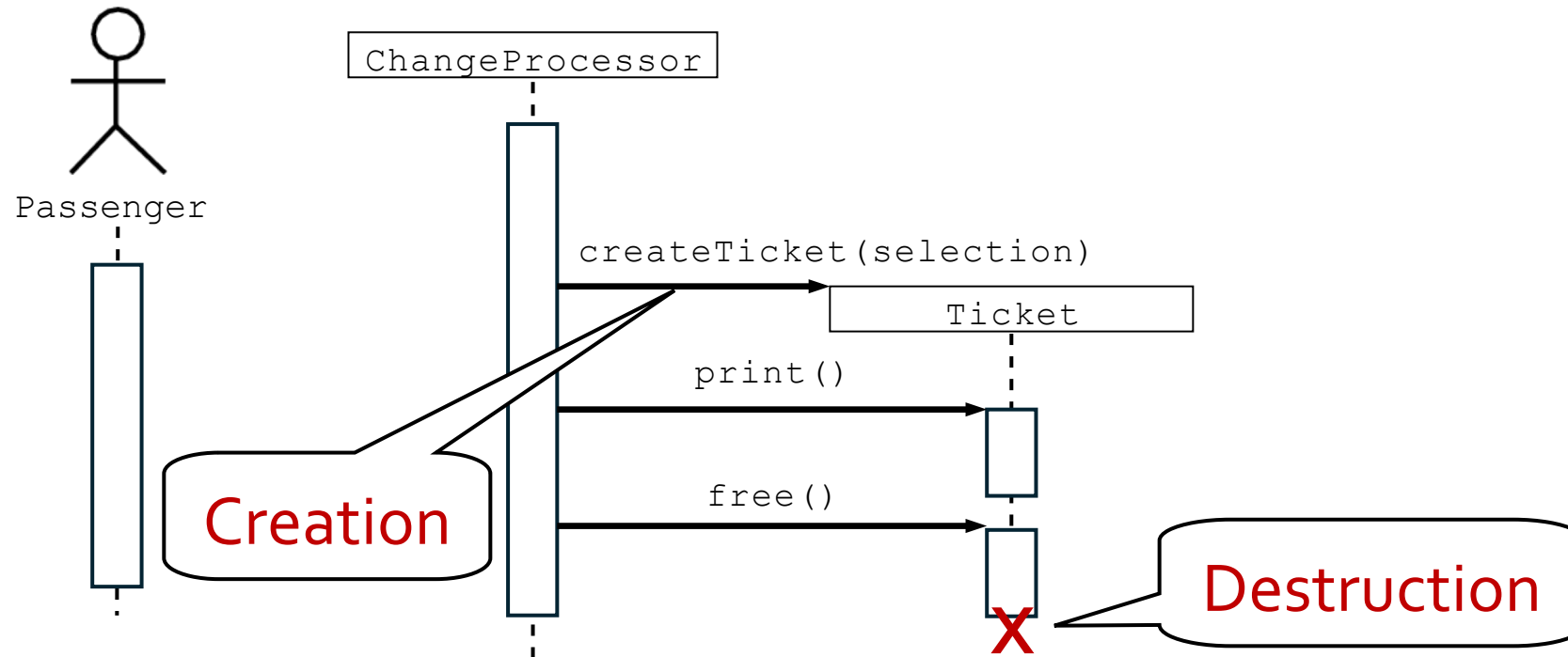
- Iteration is denoted by a * preceding the message name
- Condition is denoted by Boolean expression in [] before the message name
 - Also known as a guard

Creation and Destruction



- Creation is denoted by a message arrow *to the object*
- Destruction is denoted by an X mark at the end of the destruction activation
- In garbage collection environments, destruction can be used to denote the end of the useful life of an object

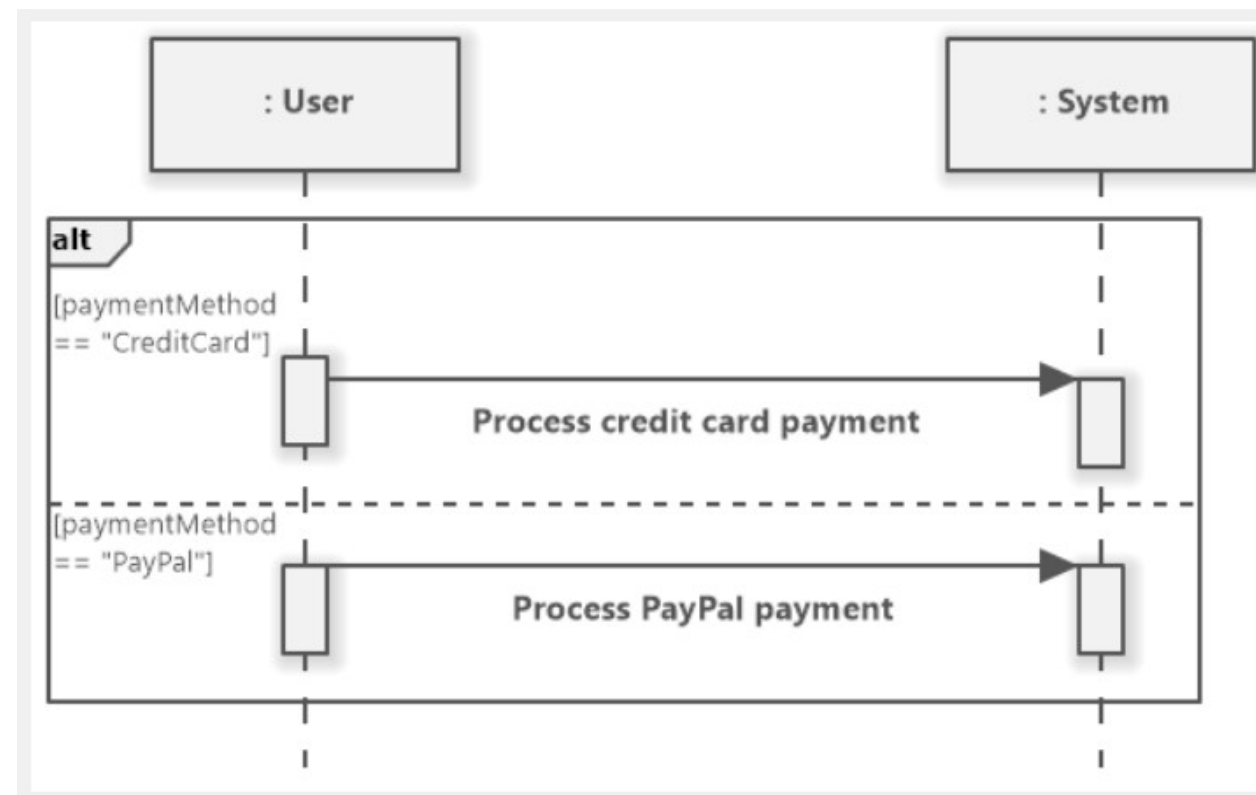
Creation and Destruction



- Creation is denoted by a message arrow *to the object*
- Destruction is denoted by an X mark at the end of the destruction activation
- In garbage collection environments, destruction can be used to denote the end of the useful life of an object

Combined Fragments: Alternative

- Model mutually exclusive *operands* (cases)
 - Only one operand will be true at a given time
 - Works like an if – else if – else statement

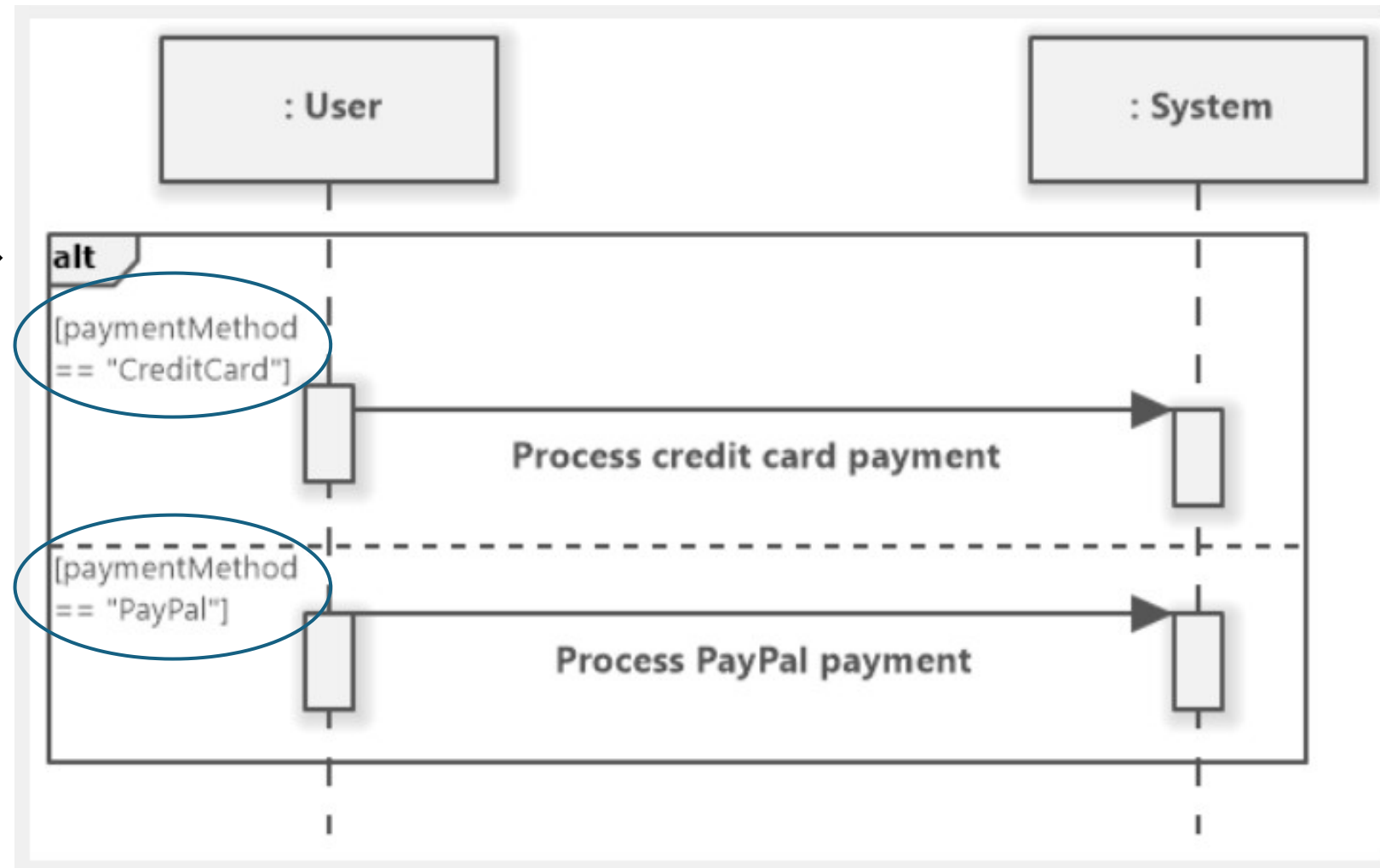


Combined Fragments: Alternative

Draw box around all operands; label box "alt" →

Dotted line to separate operands →

In this example,
the payment method
either is by credit card or
PayPal, never both.

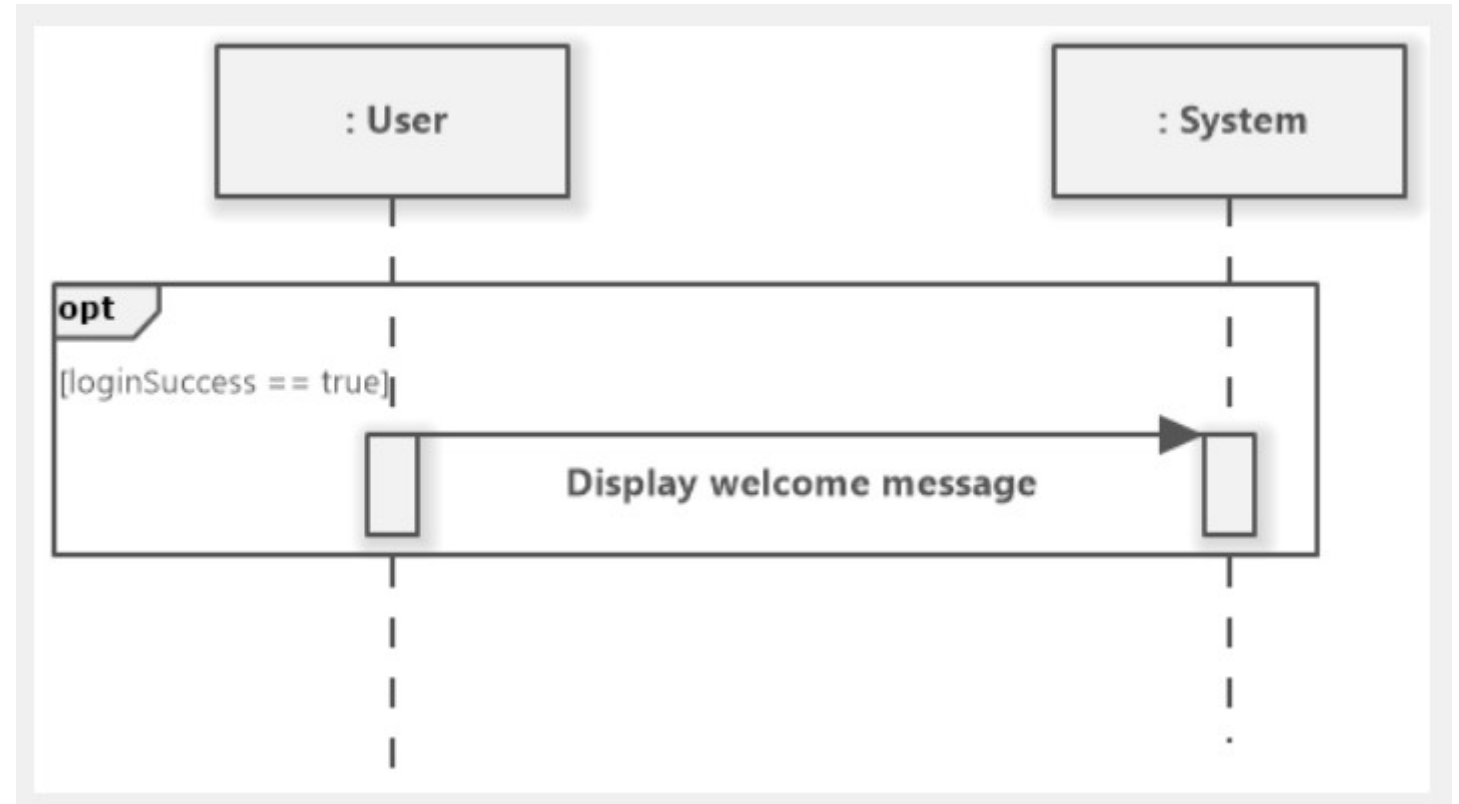


Combined Fragments: Optional

- Model a single conditional path that is optional to run

Draw box around operand; label box "opt" →

In this example, we can choose to display the welcome message if we wanted to.



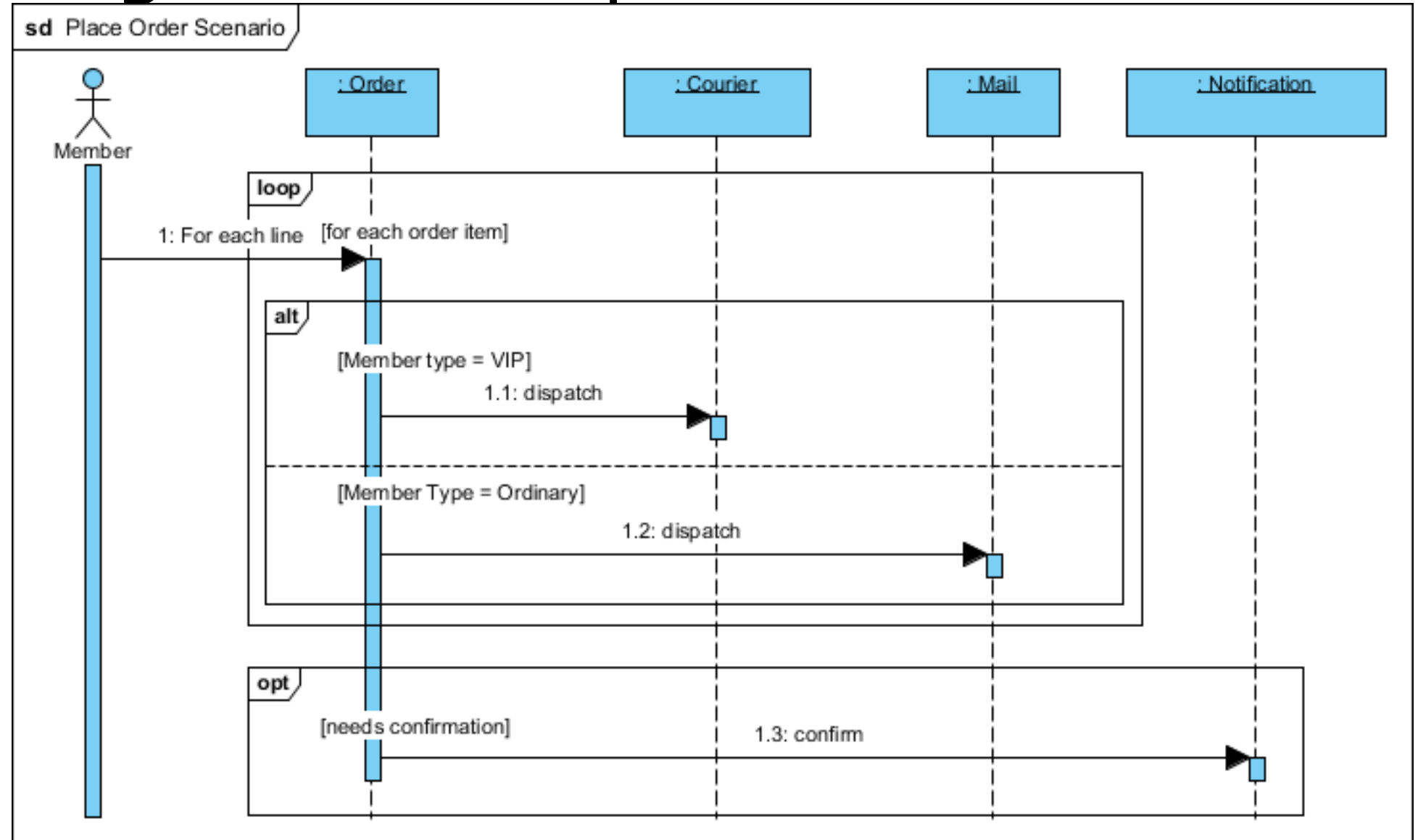
Combined Fragments: Loops

- Model repeated interaction given a condition
- Condition types
 - Iteration loops: repeat a fixed number of times
 - [i < 5]
 - While loops: repeat while a condition is true
 - [while balance > cost]
 - For each loops: iterate over a collection/list
 - [for each order item]
 - Until loops: repeat until a specific event is happens
 - [until success]

Combined Fragments: Loops

- Draw box around part to iterate over; label box "loop"
- Add condition

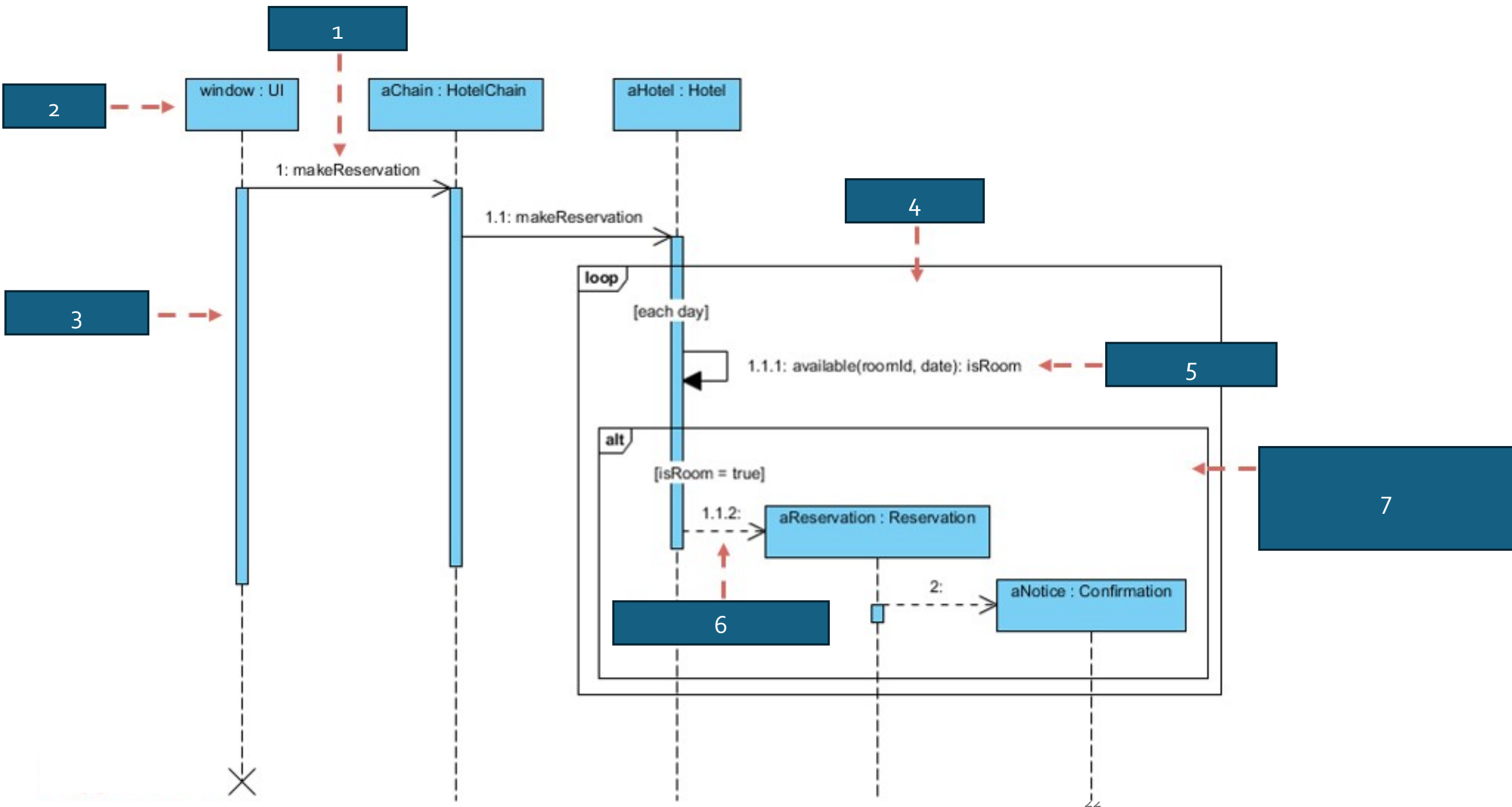
In this example, every item ordered will be checked if it will be delivered by courier or by mail

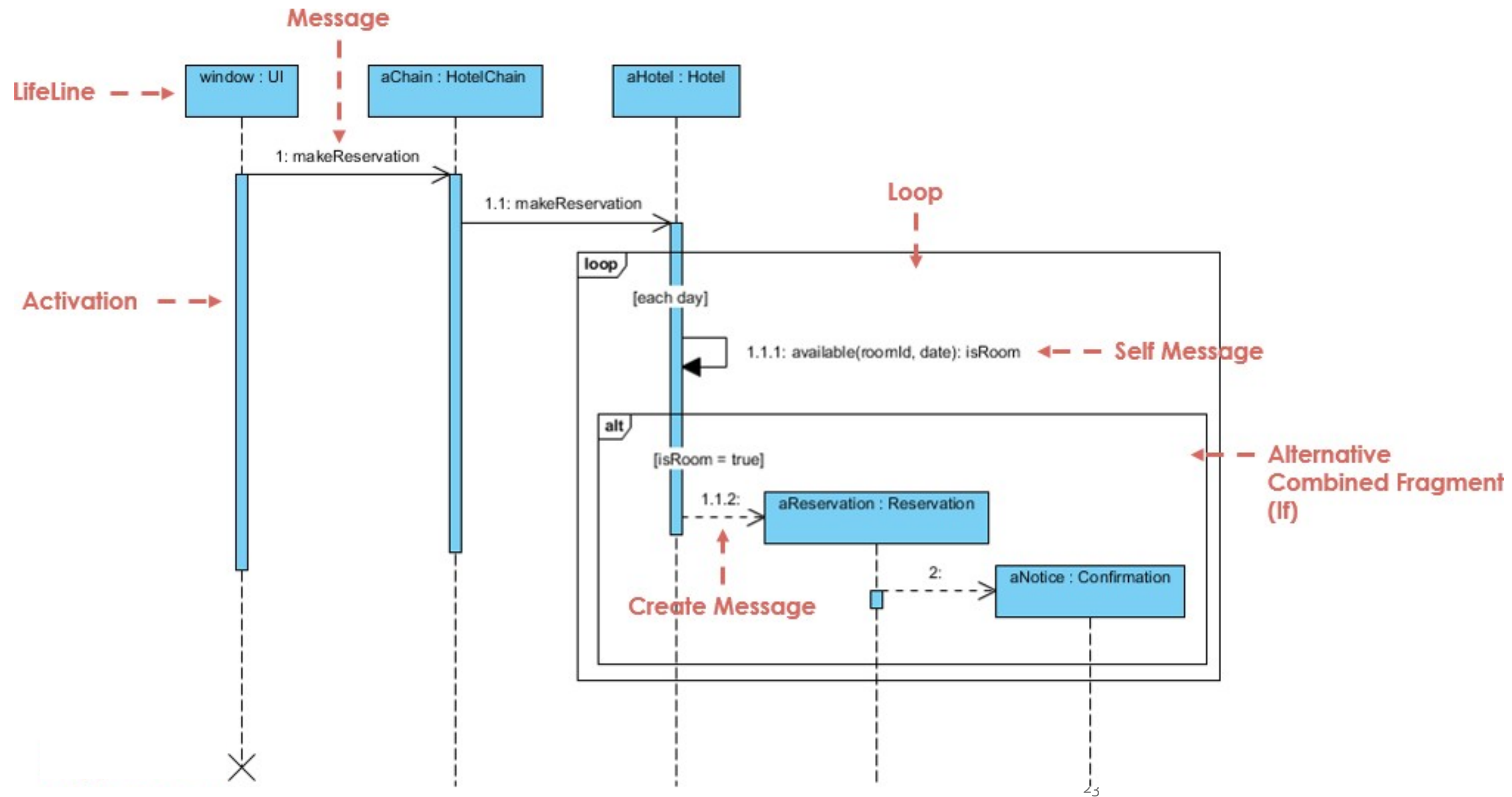


Sequence Diagram Summary

- UML sequence diagrams represent behaviour in terms of interactions
- Useful to find missing objects
- Time consuming to build but worth the investment
- Complements the class diagrams (which represent structure)

Let's Practice Reading Sequence Diagrams

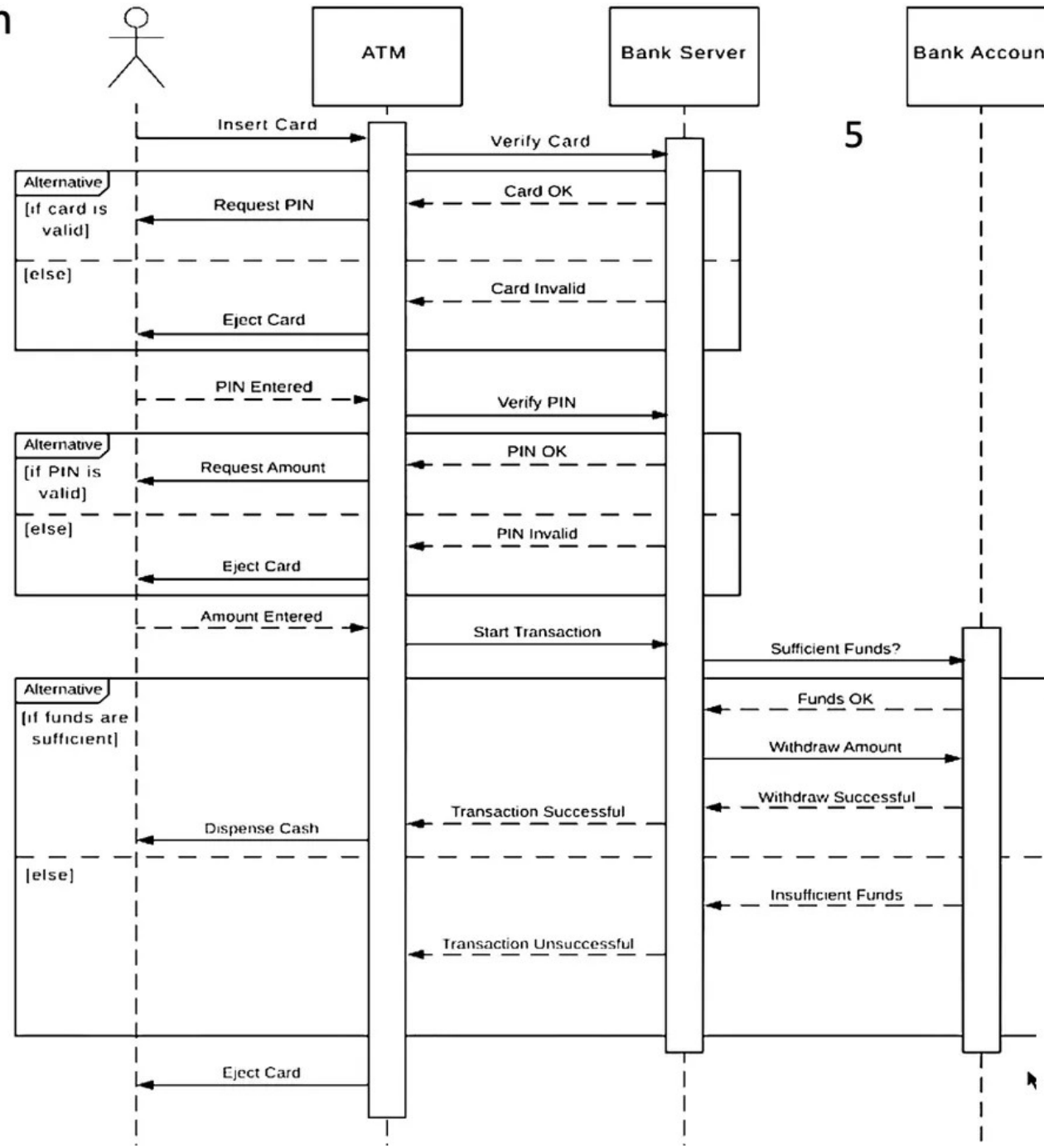


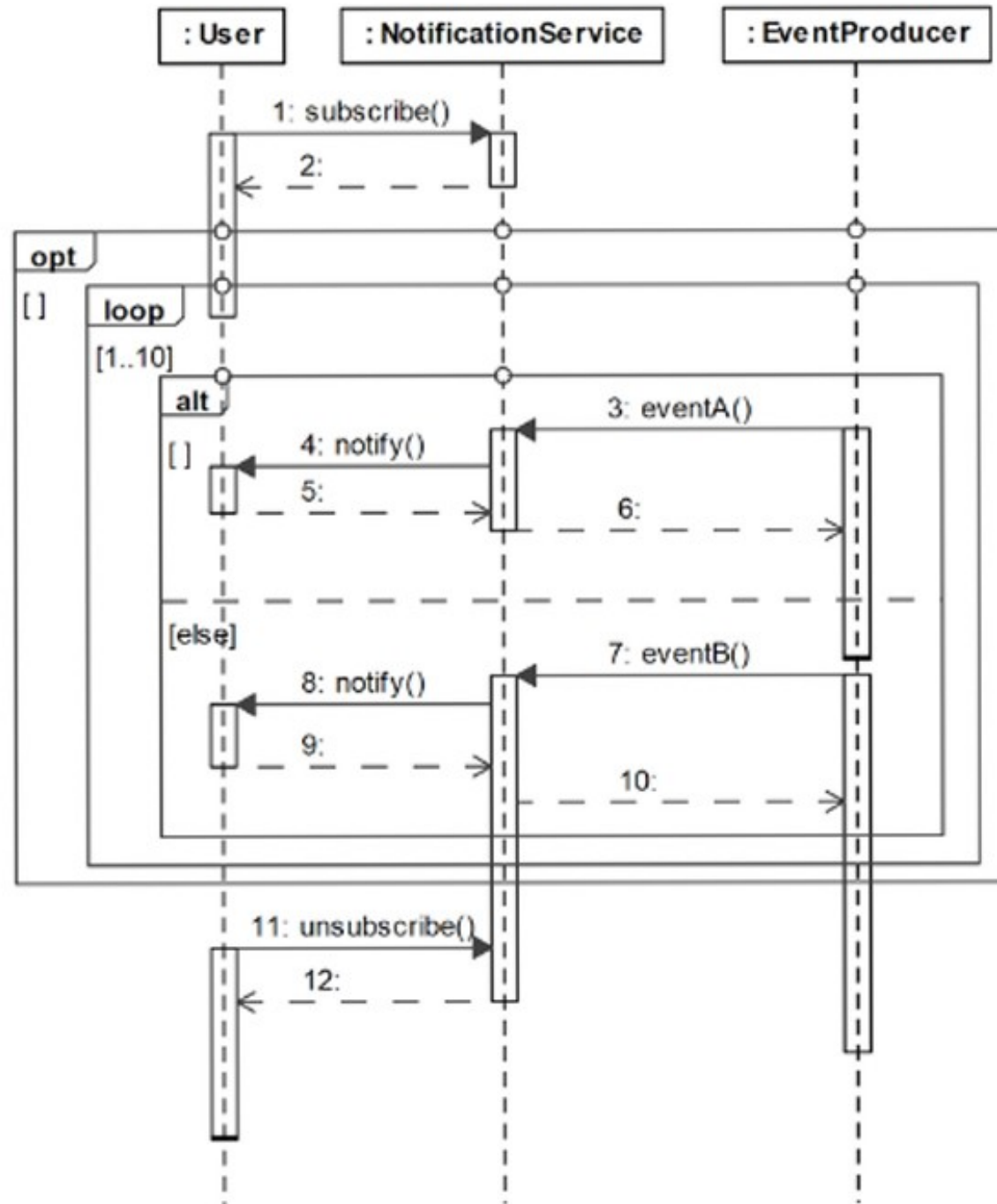
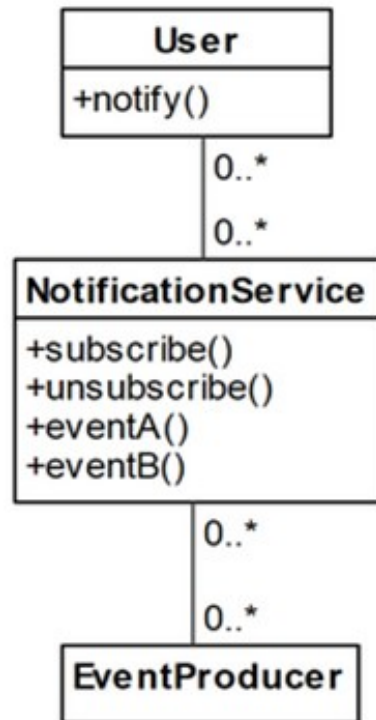


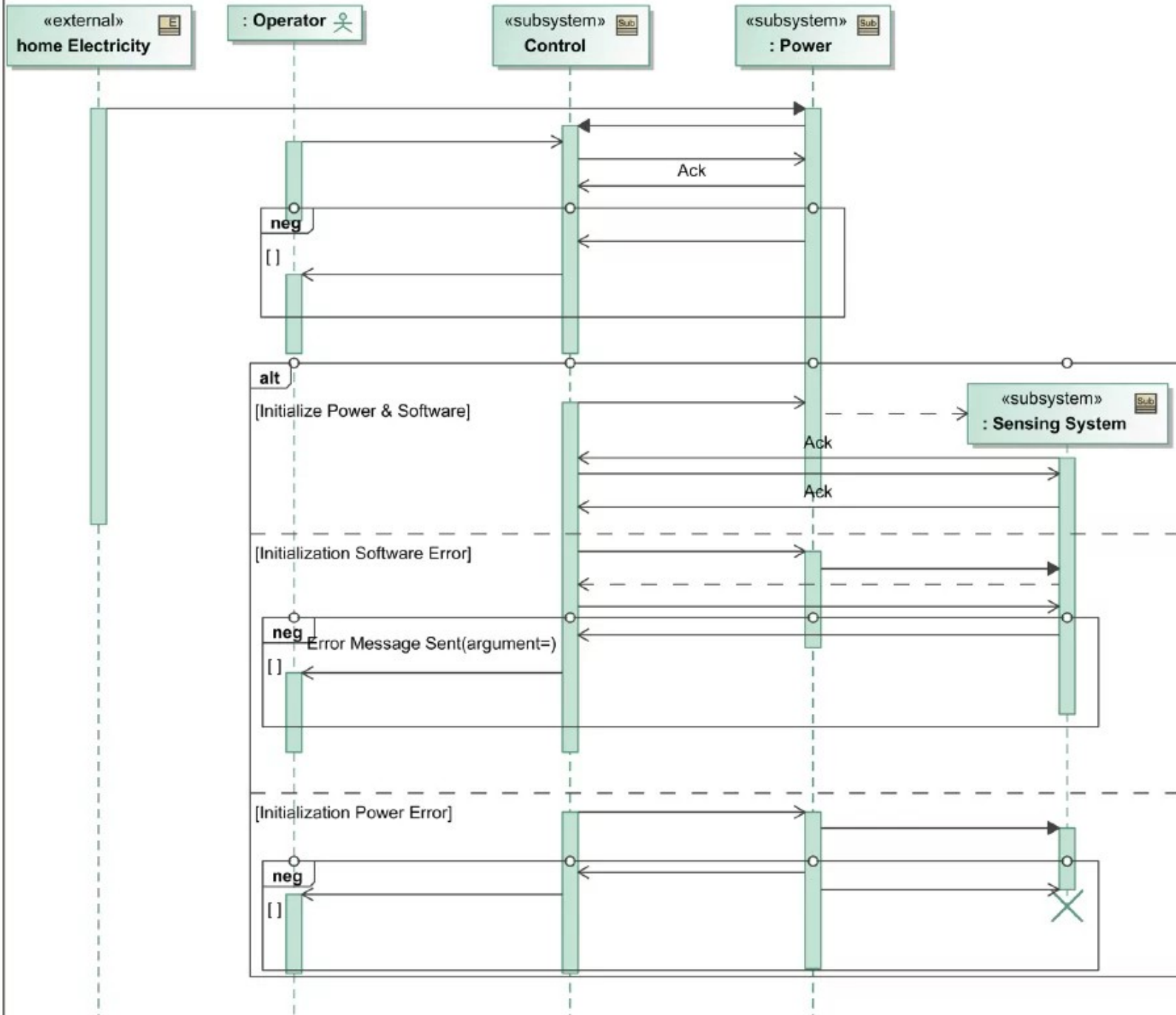


Object Dimension

Time Dimension







Spot the Mistakes

What's Wrong with This One?

Enroll In Seminar
Basic Course of Action

1. Student indicates wish to enroll

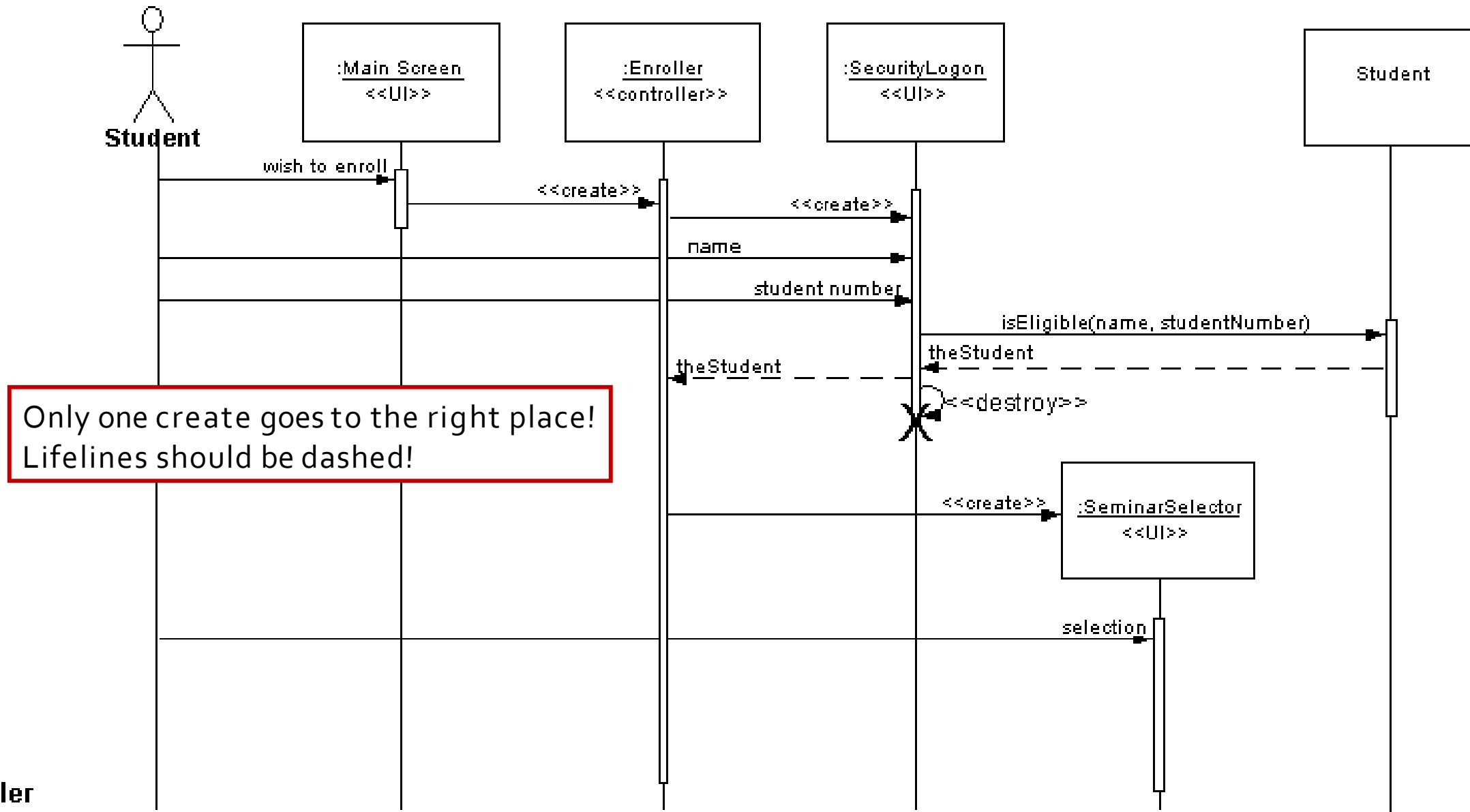
2. Student inputs name and number

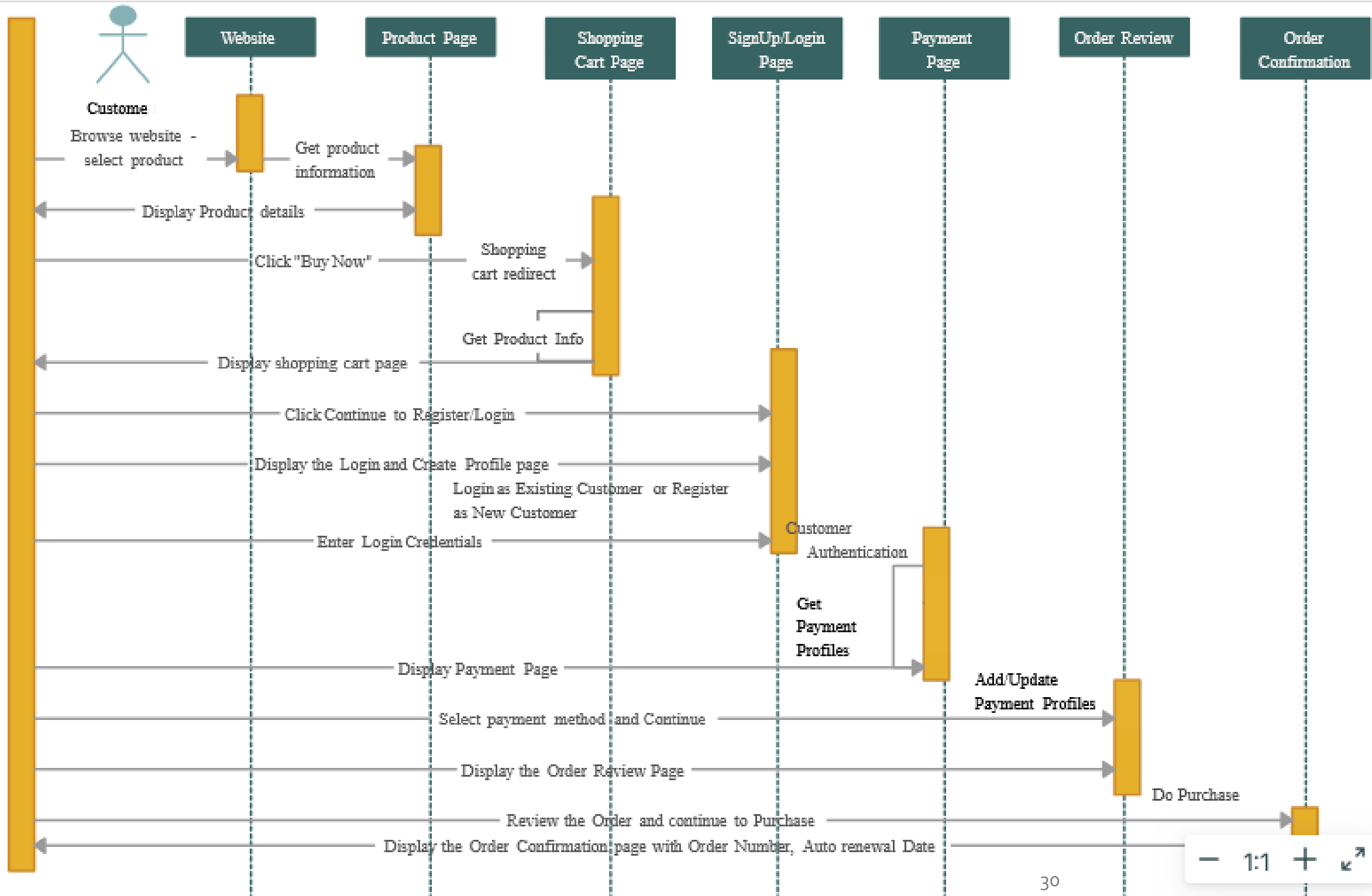
3. System verifies student

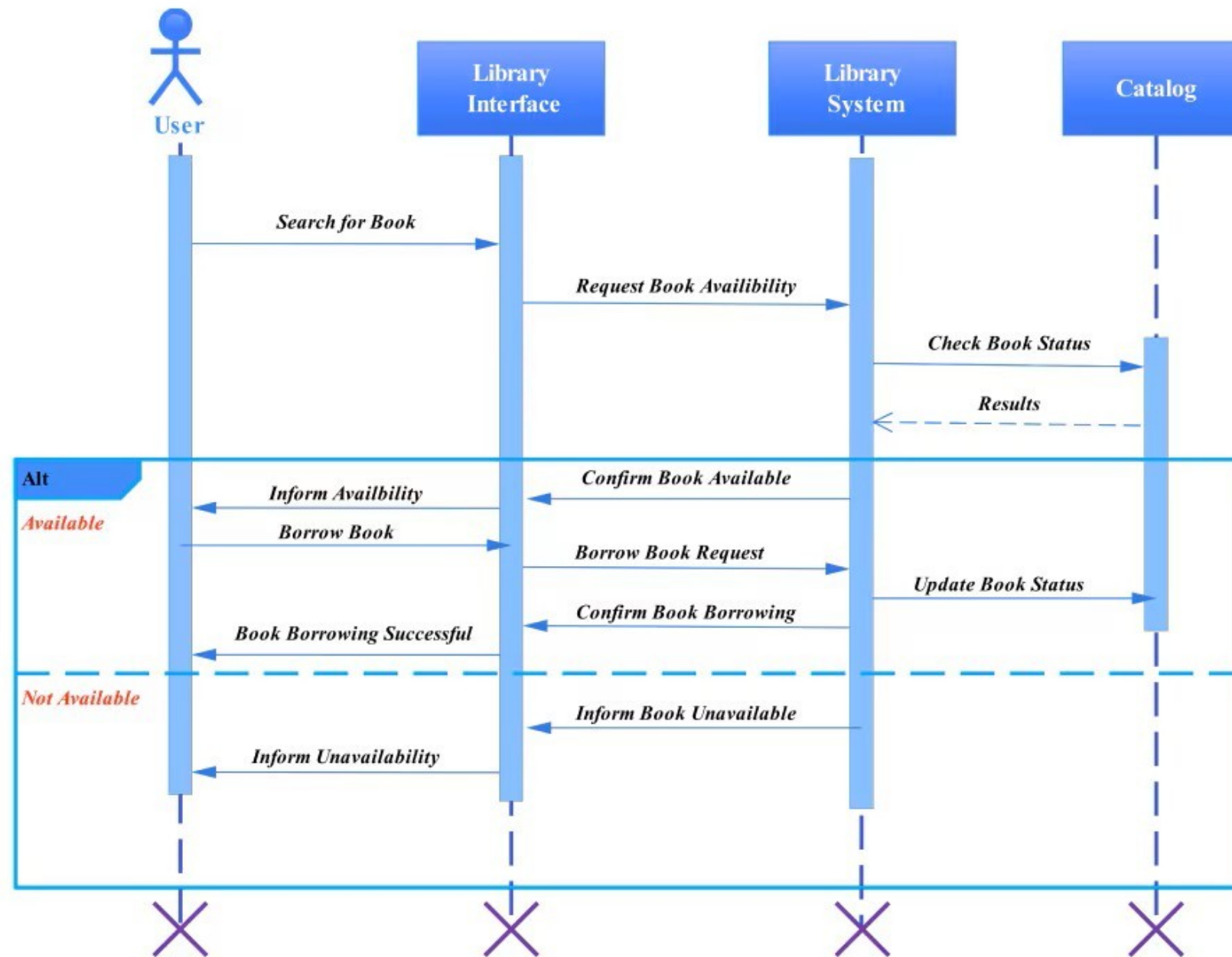
4. System displays seminar list

5. Students picks seminar

...







More Information

The IBM Page on Sequence Diagrams:

- <https://developer.ibm.com/articles/the-sequence-diagram/>

More Information

- Links:
 - UML 2 Sequence Diagrams
 - <http://agilemodeling.com/artifacts/sequenceDiagram.htm>
 - Wikipedia
 - https://en.wikipedia.org/wiki/Sequence_diagram
 - Other slides adapted from Larman et al.
 - <http://www.cse.lehigh.edu/~glennb/oose/ppt/o6SystemSequenceDiagrams.ppt>